

1. Introduccion

Behavior Driven Development (BDD) es una metodologia agil que describe el comportamiento esperado del software en lenguaje natural estructurado. Creada por Dan North (2006), responde '?que deberia hacer el sistema?' antes de '?como implementarlo?'.

Referencia principal: 'Specification by Example' (Gojko Adzic) demuestra que los ejemplos concretos son la mejor manera de comunicar comportamiento entre negocio y desarrollo. 'BDD in Action' (John Ferguson Smart) es la guia practica de implementacion.

1.1 Estructura Given-When-Then

- Given: contexto inicial (precondiciones del escenario).
- When: accion que dispara el comportamiento a probar.
- Then: resultado esperado y verificable.

Esta estructura fuerza a pensar en comportamiento observable, no implementacion interna. Un escenario BDD bien escrito es entendible por negocio y ejecutable como prueba.

1.2 BDD en el Proyecto

Escenarios Gherkin del asistente inteligente:

Feature: Asistente Inteligente

Scenario: Pregunta con respuesta en knowledge base

Given el usuario "student-01" esta registrado

When pregunta "Que es RAG?"

Then el sistema responde con contenido de knowledge_base.json

And la respuesta incluye la fuente "rag"

And context_used es una lista vacia

Scenario: Pregunta sin coincidencia

Given el usuario "student-01" esta registrado

When pregunta "Que es la teoria de cuerdas?"

Then el sistema responde que no encontro informacion

Scenario: Guardar contexto de usuario (CAG)

Given el usuario "ana" tiene sesion activa

When guarda contexto con key "preferred_style"

Then el sistema responde 201 con saved=true

2. Cobertura de Pruebas Base

Las pruebas base del proyecto validan tres escenarios fundamentales:

```
# test_base_api.py
class BaseApiTest(unittest.TestCase):

    def test_health_returns_ok(self):
        # GET /health -> 200 {"status": "ok"}
        status, body = self.get_json("/health")
        self.assertEqual(status, 200)
        self.assertEqual(body["status"], "ok")

    def test_ask_answers_from_knowledge_base(self):
        # POST /api/ask -> respuesta con sources
        status, body = self.post_json("/api/ask",
            {"user_id": "base-user", "question": "Que es RAG en el curso?"})
        self.assertEqual(status, 200)
        self.assertIn("RAG recupera", body["answer"])
        self.assertIn("rag", body["sources"])

    def test_ask_requires_user_and_question(self):
        # Sin question -> 400
        with self.assertRaises(HTTPError) as error:
            self.post_json("/api/ask", {"user_id": "base-user"})
        self.assertEqual(error.exception.code, 400)
```

Estas pruebas validan el comportamiento completo del endpoint /api/ask: caso feliz y caso error. Escribirlas ANTES de implementar el endpoint seria el enfoque BDD clasico (outside-in).

3. Pruebas de Validacion CAG

Las pruebas de validacion verifican que el comportamiento CAG funcione:

```
# test_cag_contract.py
class CagContractTest(unittest.TestCase):

    def test_saves_context_for_user(self):
        # POST /api/context -> 201 {"saved": true}
        status, body = self.post_json("/api/context",
            {"user_id": "ana", "key": "preferred_style",
             "value": "explicaciones con analogias"})
        self.assertEqual(status, 201)
        self.assertTrue(body["saved"])

    def test_retrieves_context_for_user(self):
        # GET /api/context?user_id=ana -> contexto guardado
        status, body = self.get_json("/api/context?user_id=ana")
        self.assertEqual(status, 200)
        self.assertIn({"key": "project", "value": ...}, body["context"])

    def test_ask_uses_context_to_influence_response(self):
        # Pregunta con contexto -> respuesta personalizada
        self.assertIn("principiante", body["answer"].lower())
        self.assertIn("audience", body["context_used"])
```

4. Referencias

Specification by Example (Gojko Adzic) - Manning Publications

BDD in Action (John Ferguson Smart) - Manning Publications

Cucumber Documentation - cucumber.io

Behave (Python BDD) - behave.readthedocs.io

Gherkin Reference - cucumber.io/docs/gherkin